LIBRARY MANAGEMENT SYSTEM

Using Java (OOP & Swing)

Capstone Project Report

Submitted By:

- Akshat Sharma — SAP ID: 590011605
- Manan Rajoria — SAP ID: 590012746
- Samay Chutani — SAP ID: 590011697

Course Details

Course Name: Btech Computer Science
Batch: 10, 11, 10

Submitted To:

Faculty Name: Deepika Sachdev
School of Computer Science
Submission Date: 1st May 2026

1. Project Overview

1.1 Problem Statement

Traditional library systems rely on manual record-keeping, which makes it difficult to manage books, borrowers, and due dates efficiently. This often leads to errors, delays, and poor tracking.

This project solves these issues by developing a Library Management System using Java Swing that automates book management and borrowing processes.

1.2 Scope

The system allows:

- Adding, editing, and deleting books
- Borrowing and returning books
- Tracking borrower details
- Managing book quantity
- Detecting overdue books

1.3 Objectives

- To develop a library system using Java and OOP
- To manage books and borrower records
- To implement due-date tracking
- To create a user-friendly GUI

2. Requirement Analysis

2.1 Functional Requirements

What the system should do:

- Add, edit, and delete books
- Display books in table
- Borrow and return books
- Update quantity automatically
- Store borrower details
- Generate borrow date and due date
- Show overdue alerts

2.2 Non-Functional Requirements

- Easy to use interface
- Reliable and error-free
- Fast performance
- Simple and maintainable code

2.3 Software Requirements

Component	Details
Language	Java
IDE	VS Code / IntelliJ
GUI	Swing
Database	Not used

3. System Design

3.1 Module Description

Book Module
Handles book details like title, author, section, and quantity.

Borrow Module
Handles borrowing and returning of books.

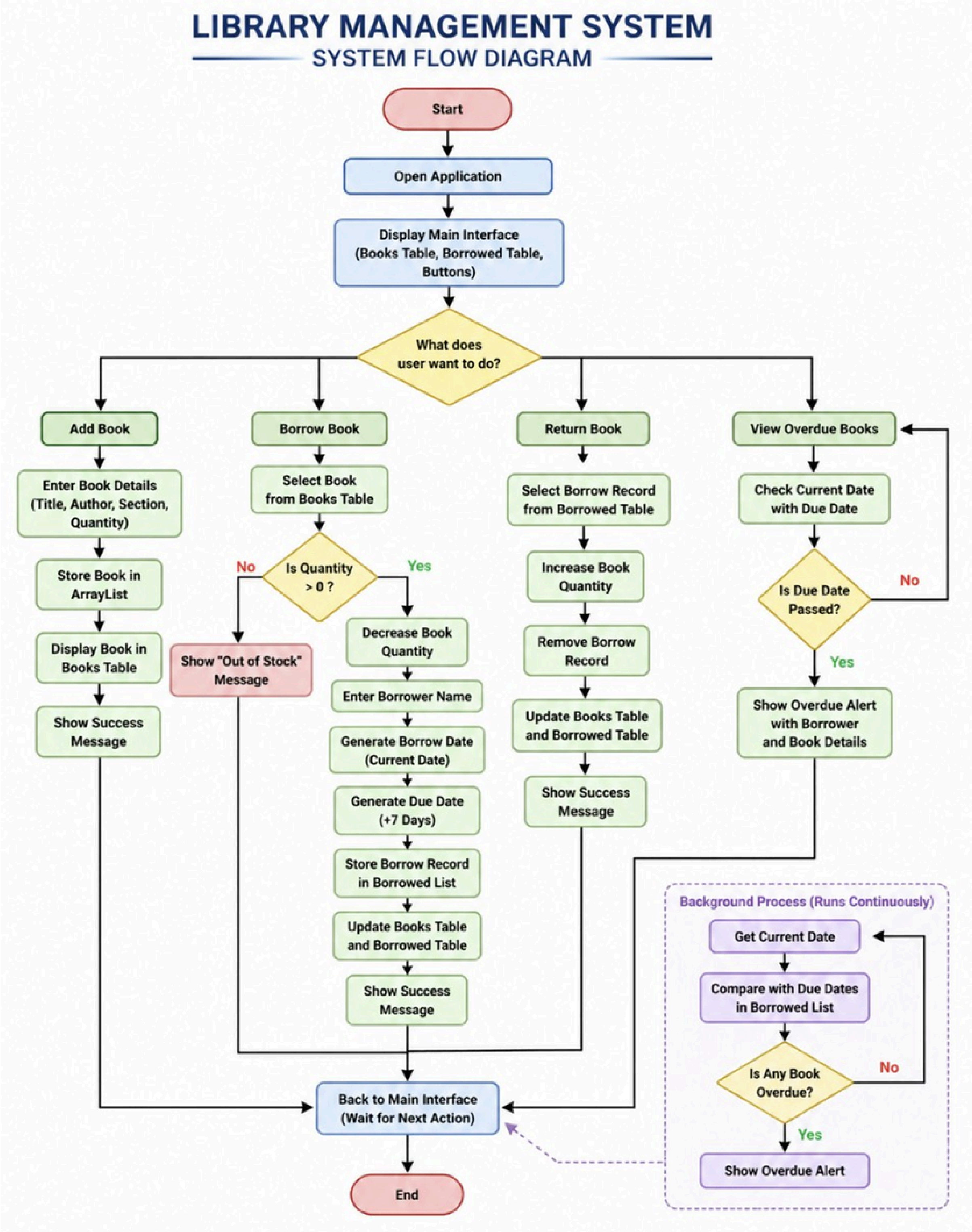
UI Module
Provides a graphical interface using Swing.

3.2 System Flow

The system follows this flow:

- Add Book
- Display in Table
- Select Book
- Borrow Book
- Record Date & Due Date
- Return Book
- Show Overdue Alert

Figure 2: System Flow Diagram



The diagram shows the working flow of the library system.

3.3 Data Structure Design

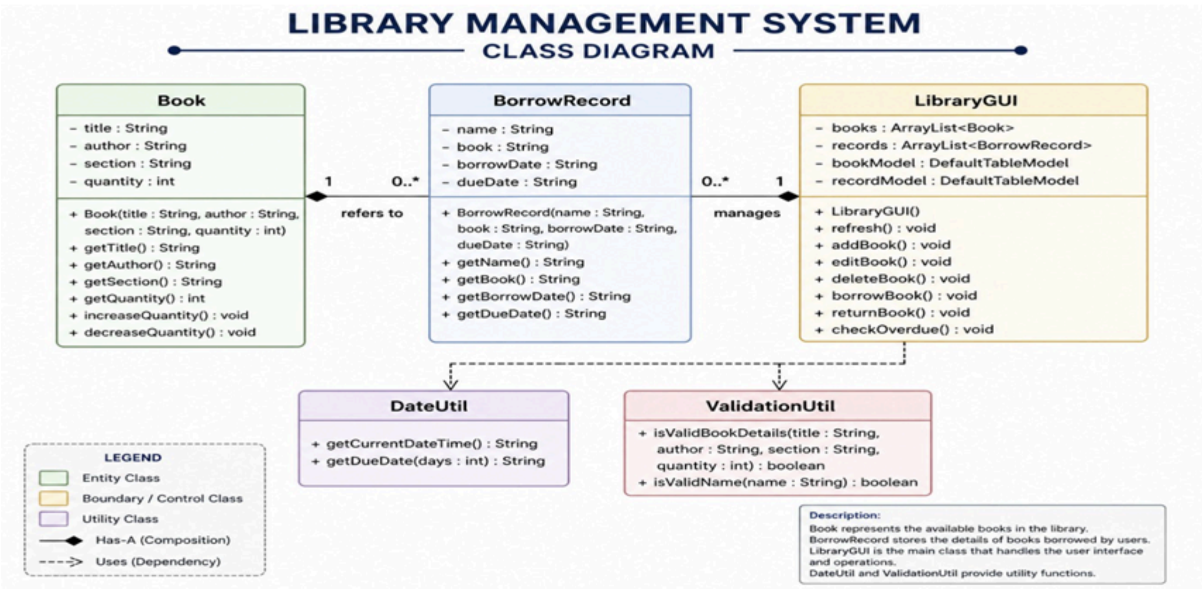
Book Class

Field	Type
title	String
author	String
section	String
quantity	int

BorrowRecord Class

Field	Type
name	String
book	String
borrowDate	String
dueDate	String

Figure 3: Class Diagram



The diagram shows the structure of Book and BorrowRecord classes.

4. Implementation

4.1 Technologies Used

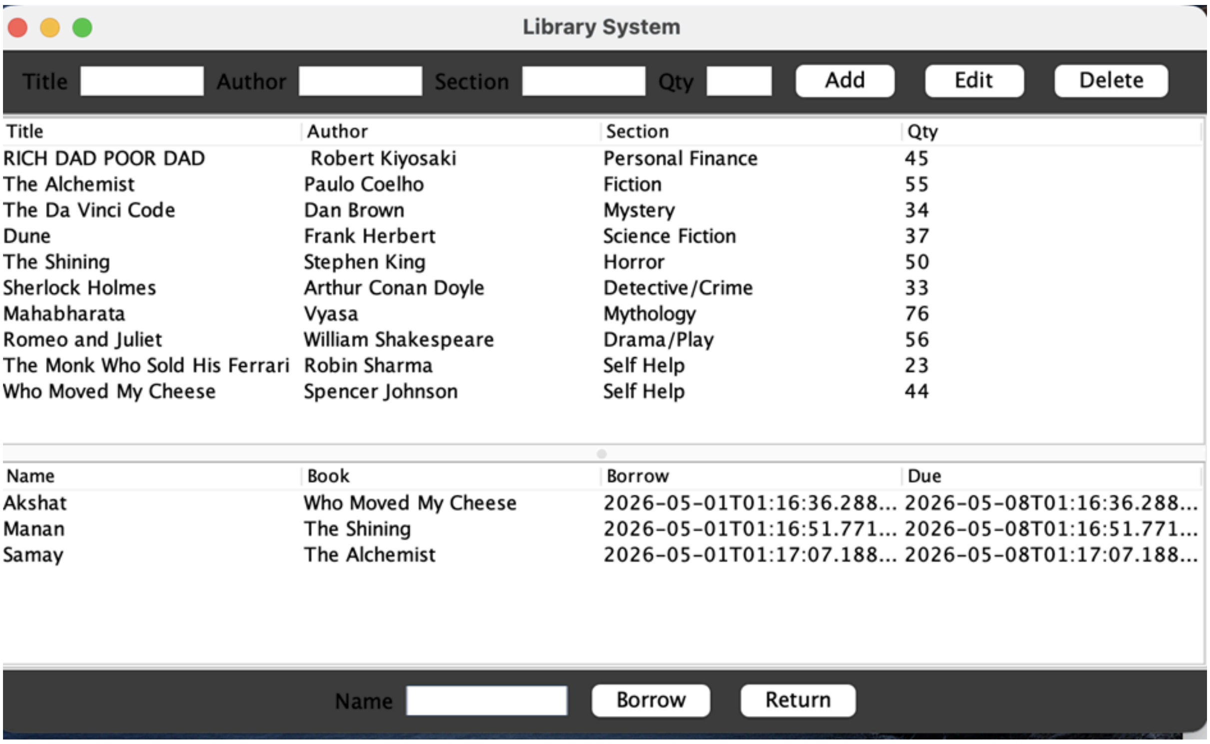
Technology	Purpose
Java	Programming
Swing	GUI
ArrayList	Data storage
Date-Time API	Due date

4.2 Project Structure

```
LibraryProject/
├── Book.java
├── BorrowRecord.java
├── LibraryGUI.java
└── Main.java
```

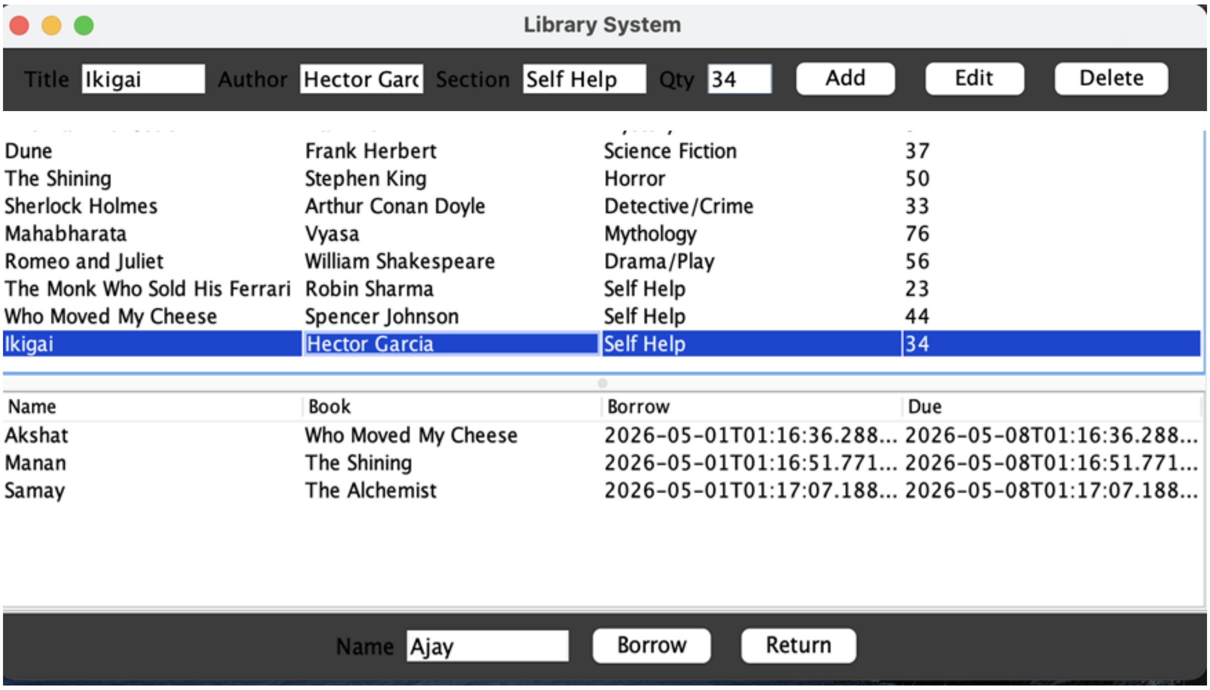
4.3 Screenshots of GUI

Figure 1: Main Interface



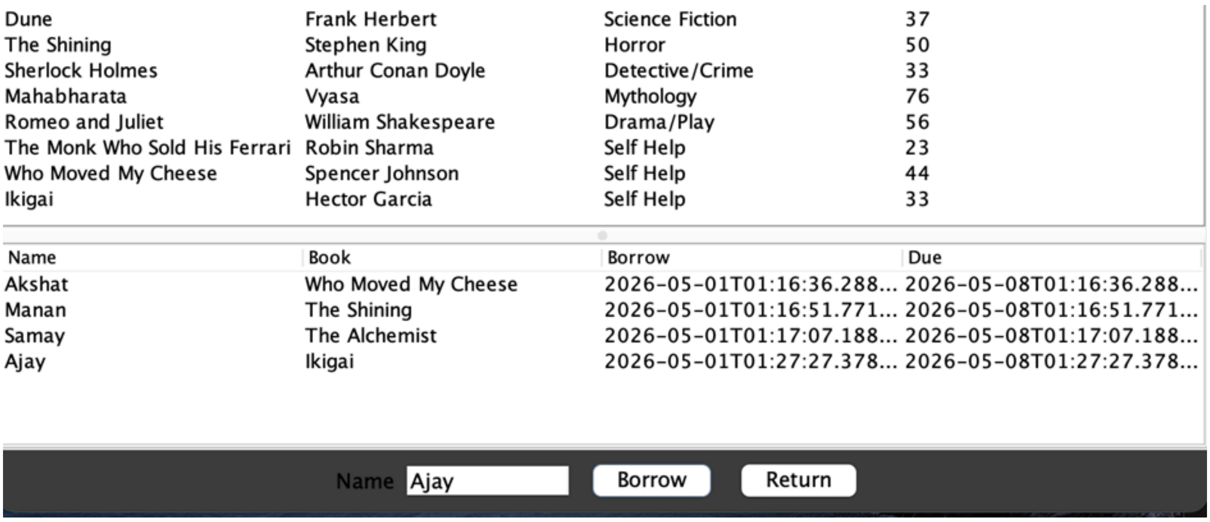
The above figure shows the main interface of the Library Management System.

Figure 2: Book Selection and Editing



The figure shows a selected book from the table where the details are automatically filled into input fields, allowing the user to edit or update the book information.

Figure 3: Borrow Records Display



The figure shows the borrower records including user name, borrowed book, borrow date, and due date, maintained by the system.

4.4 Key Code Snippets

Add Book:

```
books.add(new Book(
    t.getText(), a.getText(),
    s.getText(), Integer.parseInt(q.getText())
));
```

Borrow Book:

```
books.get(i).quantity--;
LocalDateTime now = LocalDateTime.now();
LocalDateTime due = now.plusDays(7);
records.add(new BorrowRecord(
    name.getText(), books.get(i).title,
    now.toString(), due.toString()
));
```

Return Book:

```
for(Book b: books){
    if(b.title.equals(bookName)){
        b.quantity++;
        break;
    }
}
records.remove(i);
```

Overdue Detection:

```
if(now.isAfter(LocalDateTime.parse(r.dueDate))){
    JOptionPane.showMessageDialog(this,
        "Overdue! " + r.name + " has not returned " + r.book);
}
```

5. OOP Concepts Applied

Concept	Explanation
Encapsulation	Data stored in classes
Abstraction	Simplified design
Collections	Used ArrayList
Event Handling	Button actions

6. GitHub and Version Control

6.1 Repository Link

https://github.com/Manan366/LibraryManagementSystem

7. Advantages

- Easy to use
- Fast processing
- Automatic tracking
- Reduces manual work

8. Limitations

- No database
- No login system
- Data not saved permanently

9. Future Enhancements

- Add database (MySQL)
- Add login system
- Add fine calculation
- Add search feature

10. Conclusion

This project successfully implements a Library Management System using Java Swing. It demonstrates OOP concepts, GUI design, and real-time book tracking.

This project implements a Library Management System using Java Swing with features like book management, borrower tracking, due-date monitoring, and overdue alerts.